Computational Thermofluid Dynamics - Chapter 6

Technical University of Munich, Professur für Thermofluiddynamik - Pr. Polifke

Created: 12/2024 | J. Yao, N. Garcia, G. Varillon

Revised: 12/2025 | A. Keles

==========

Stage 1 - Numerical Integration

Before implementing Green's functions, it is useful first to refresh our minds about numerical integration with the function trapezoid. Perform $\int_0^2 x dx$ and $\int_0^2 \int_0^2 \cos(x_s)\sin(y_s) dx_s dy_s$ and print the result. use 1000 point

In [6]:
```python
import numpy as np


def f(x):
    return x

def g(x, y):
    return np.cos(x)*np.sin(y)

# Your solution
x = np.linspace(0, 2, 1000)
y = np.linspace(0, 2, 1000)

X, Y = np.meshgrid(x, y)
G = g(X, Y)

int_f = np.trapezoid(f(x), x)
int_g_x = np.trapezoid(G, x, axis=1)
int_g = np.trapezoid(int_g_x, y)
print(f"Integration of the f function: {int_f:.5f}")
print(f"Integration of the g function: {int_g:.5f}")
```

```
Integration of the f function: 2.00000
Integration of the g function: 1.28770
```

Stage 2 – 1D Green Function

In this stage we construct the 1D Green's function for the 1D steady heat equation using eigenfunction expansion.

For boundary case **X11 (Dirichlet–Dirichlet)**:

$$X_m(x) = \sin\left(\frac{m\pi x}{L}\right), \qquad \beta_m = \frac{m\pi}{L}, \qquad N_x = \frac{L}{2}.$$

The truncated Green's function is

$$G_{1D}(x \mid x_s) = \sum_{m=1}^{M} \frac{X_m(x)\, X_m(x_s)}{N_x\, \beta_m^2}.$$

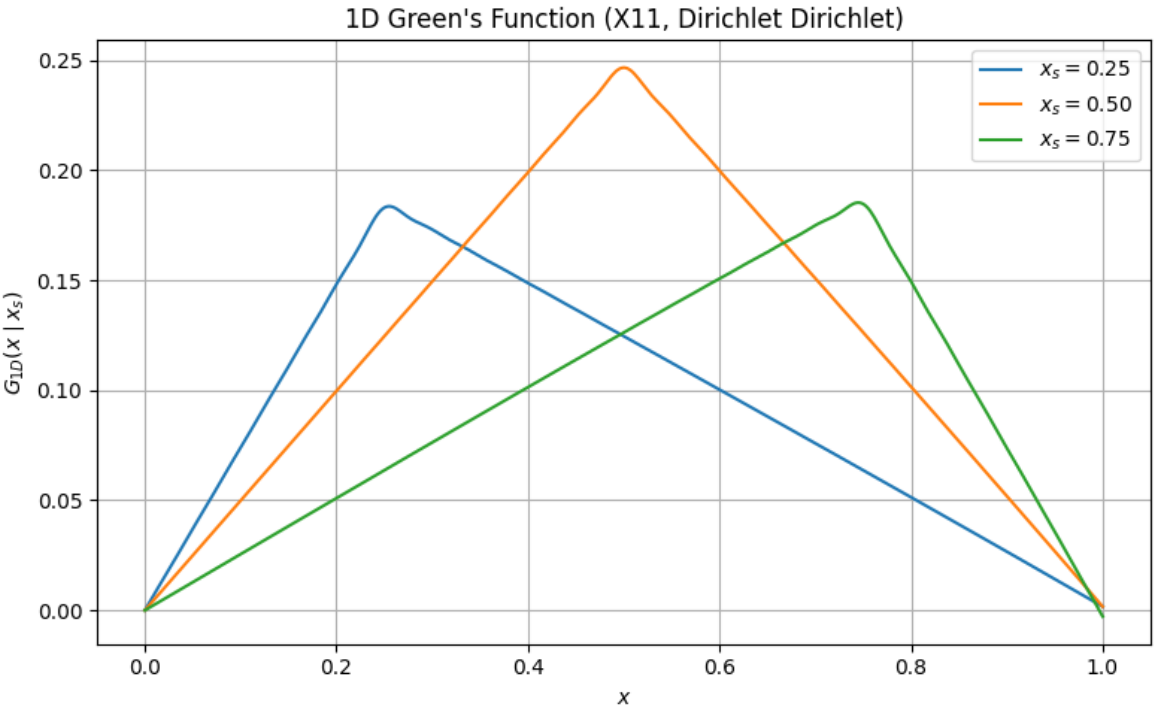
Tasks:

1. Implement $X_m(x)$, β_m, and N_x.
2. Implement $G_{1D}(x \mid x_s)$ with $M = 50$.
3. Evaluate and plot the function for $x_s = 0.25L,\ 0.5L,\ 0.75L$.

In [7]:

```
import numpy as np
import matplotlib.pyplot as plt

# Domain length
L = 1

def X_m(m, x, L):
    return np.sin(m * np.pi * x / L)

def beta_m(m, L):
    return m * np.pi / L

def Nx(L):
    return L/2

def G_1D(x, xs, L, M=50):
    """
    Compute truncated 1D Green's function G_1D(x | xs)
    for X11 boundary conditions with M modes.
    x  : observation points (array)
    xs : source location (scalar)
    """
    m = np.linspace(1, M+1, M)
    x_m = np.ones((len(x), M)) * x.reshape(-1,1)
    xs_m = X_m(m, xs, L)
    nx = Nx(L)
    bm = beta_m(m, L)
    green = np.ones_like(x_m)
    for i in range(x_m.shape[0]):
        green[i, :] = X_m(m, x_m[i, :], L) * xs_m / nx / np.pow(bm, 2)
    return np.sum(green, axis=1)


# Discretize x
x = np.linspace(0, L, 500)

# Source locations
xs_list = [0.25 * L, 0.5 * L, 0.75 * L]

plt.figure(figsize=(8, 5))

for xs in xs_list:
    G_vals = G_1D(x, xs, L, M=50)
```

```
    plt.plot(x, G_vals, label=fr"$x_s = {xs:.2f}$")

plt.xlabel(r"$x$")
plt.ylabel(r"$G_{1D}(x \mid x_s)$")
plt.title("1D Green's Function (X11, Dirichlet Dirichlet)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



Stage 3 – 2D Green Function (X11 and X12 Cases)

In this stage you will construct the 2D Green's function

$$G(x, y \mid x_s, y_s)$$

for two different sets of boundary conditions: **X11** (Dirichlet–Dirichlet) and **X12** (Dirichlet–Neumann).

Domain Setup

Use a rectangular domain

$$0 \le x \le L_x, \qquad 0 \le y \le L_y,$$

with

$$L_x = 1, \qquad L_y = 1.$$

Discretize the domain into a mesh:

$$x = \text{linspace}(0, L_x, N_x), \qquad y = \text{linspace}(0, L_y, N_y).$$

Use

$$N_x = N_y = 100.$$

Source Location

Choose a **single point source** (delta source) located at

$$(x_s, y_s) = (0.5,\ 0.5).$$

This point is used inside $G(x, y \mid x_s, y_s)$.

Green Function Form

The 2D Green's function is given by the mode expansion

$$G(x, y \mid x_s, y_s) = \sum_{m=1}^{M} \sum_{n=1}^{N} \frac{X_m(x)\, X_m(x_s)\, Y_n(y)\, Y_n(y_s)}{N_x N_y \left(\beta_m^2 + \theta_n^2\right)}.$$

Use

$$M = N = 30.$$

X11 Case (Dirichlet–Dirichlet)

Eigenfunctions:

$$X_m(x) = \sin\left(\frac{m\pi x}{L_x}\right), \qquad Y_n(y) = \sin\left(\frac{n\pi y}{L_y}\right).$$

Eigenvalues:

$$\beta_m = \frac{m\pi}{L_x}, \qquad \theta_n = \frac{n\pi}{L_y}.$$

Normalization:

$$N_x = \frac{L_x}{2}, \qquad N_y = \frac{L_y}{2}.$$

X12 Case (Dirichlet–Neumann)

Eigenfunctions:

$$X_m(x) = \sin\left(\frac{(2m-1)\pi x}{2L_x}\right), \qquad Y_n(y) = \sin\left(\frac{(2n-1)\pi y}{2L_y}\right).$$

Eigenvalues:

$$\beta_m = \frac{(2m-1)\pi}{2L_x}, \qquad \theta_n = \frac{(2n-1)\pi}{2L_y}.$$

Normalization remains:

$$N_x = \frac{L_x}{2}, \qquad N_y = \frac{L_y}{2}.$$

Tasks

1. Implement eigenfunctions and eigenvalues for **both X11 and X12**.
2. Construct
   - $G_{11}(x, y \mid x_s, y_s)$
   - $G_{12}(x, y \mid x_s, y_s)$

   using the series above.
3. Evaluate both Green functions on the (x, y) grid.
4. Produce two plots:
   - 2D Green function for X11
   - 2D Green function for X12
5. Compare the shapes of the two Green functions.

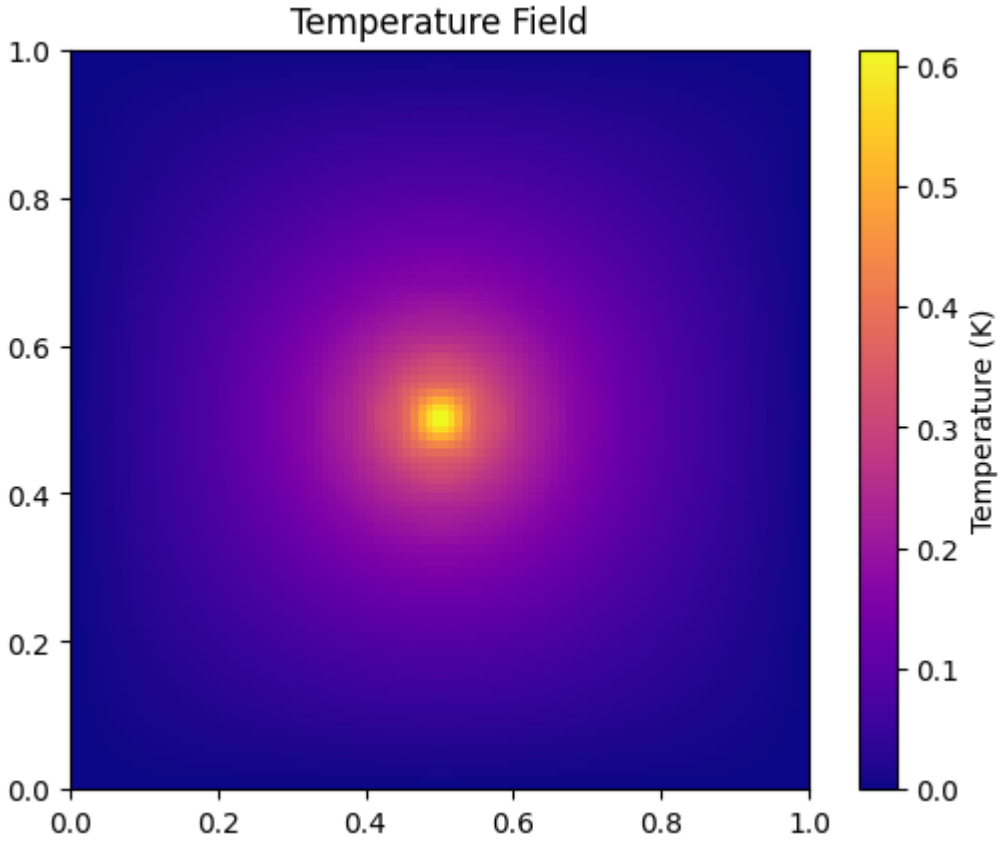
In [8]:

```python
import numpy as np
import matplotlib.pyplot as plt

# ------------------------------
# Domain and mesh
# ------------------------------
Lx, Ly = 1.0, 1.0
Nx, Ny = 100, 100

x = np.linspace(0, Lx, Nx)
y = np.linspace(0, Ly, Ny)
X, Y = np.meshgrid(x, y)

# Source point (single point)
xs, ys = 0.5, 0.5


# ------------------------------
# X11 CASE (Dirichlet–Dirichlet)
# ------------------------------

def Xm_11(m, x, L):
    return np.sin(m * np.pi * x / L)

def beta_11(m, L):
    return m * np.pi / L

Nx_norm = Lx/2
Ny_norm = Ly/2


def G_2D_X11(X, Y, xs, ys, Lx, Ly, M=30, N=30):
```

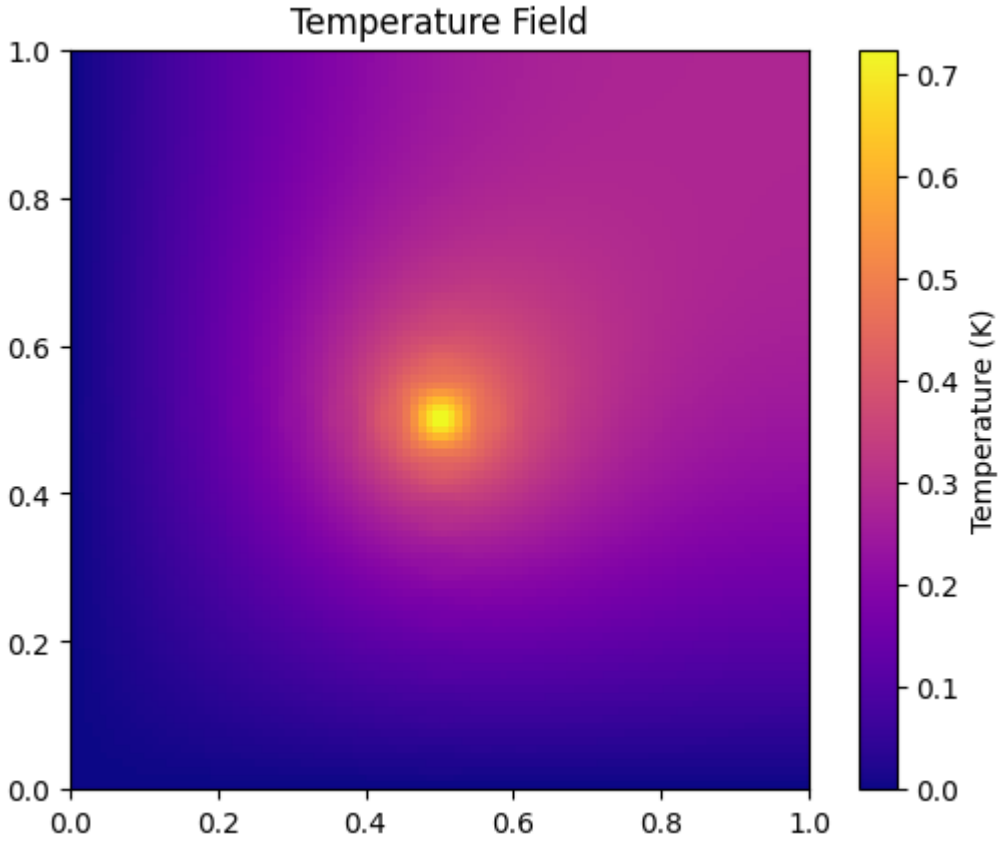
```
    x_m = np.zeros_like(X)
    y_m = np.zeros_like(Y)
    green = np.zeros_like(X)
    for m in range(1, M+1):
        x_m = Xm_11(m, X, Lx) * X_m(m, xs, Lx)
        for n in range(1, N+1):
            y_m = Xm_11(n, Y, Ly) * X_m(n, ys, Ly)
            green += x_m * y_m / (Nx_norm * Ny_norm *(beta_11(m, Lx)**2 +
    return green

# -------------------------------
# X12 CASE (Dirichlet–Neumann)
# -------------------------------

def Xm_12(m, x, L):
    return np.sin((2*m - 1) * np.pi * x / (2 * L))

def beta_12(m, L):
    return (2 * m - 1) * np.pi / (2 * L)

Nx_norm_12 = Lx / 2
Ny_norm_12 = Ly / 2

def G_2D_X12(X, Y, xs, ys, Lx, Ly, M=30, N=30):
    x_m = np.zeros_like(X)
    y_m = np.zeros_like(Y)
    green = np.zeros_like(X)
    for m in range(1, M+1):
        x_m = Xm_12(m, X, Lx) * Xm_12(m, xs, Lx)
        for n in range(1, N+1):
            y_m = Xm_12(n, Y, Ly) * Xm_12(n, ys, Ly)
            green += x_m * y_m / (Nx_norm_12 * Ny_norm_12 *(beta_12(m, Lx
    return green

def plot(TemperatureField):
    """
    Visualize the temperature field as a 2D contour or heatmap.
    """
    plt.imshow(TemperatureField,
               cmap='plasma',
               interpolation='none',
               origin='lower',
               extent=(0, Lx, 0, Ly),
               aspect='equal')
    plt.colorbar(label='Temperature (K)')
    plt.title('Temperature Field')
    plt.show()

# -------------------------------
# Compute both Green functions
# -------------------------------
G11 = G_2D_X11(X, Y, xs, ys, Lx, Ly)
G12 = G_2D_X12(X, Y, xs, ys, Lx, Ly)


# -------------------------------
# Plot results
# -------------------------------
print("Dirichlet–Dirichlet Green function:")
plot(G11)
```

```
print("Dirichlet–Neumann Green function:")
plot(G12)
```

```
Dirichlet–Dirichlet Green function:
```




```
Dirichlet–Neumann Green function:
```



Stage 4 – Superposition: Finite Difference

+ Green's Function (Point Source)

In this stage we compute the full temperature field by combining:

1. A **homogeneous solution** from the finite-difference solver:

$$-\nabla^2 T_h = 0 \qquad \text{with boundary conditions}$$

2. A **particular solution** due to a *point source* of strength (q) located at

$$(x_s, y_s) = (0.5,\ 0.5).$$

   From Stage 3, the Green's function for a point source gives:

$$T_p(x, y) = q\, G(x, y \mid x_s, y_s).$$

The total solution is obtained by superposition:

$$T(x, y) = T_h(x, y) + T_p(x, y).$$

Tasks

1. Choose boundary conditions for two cases

- **Case X11:** All four sides Dirichlet with (T = T_d).
- **Case X12:** Two sides Dirichlet and two sides Neumann (e.g. south & west Dirichlet, north & east Neumann).

Solve each case **without any internal source** to obtain the homogeneous field:

```
heat = SteadyHeat2D(Lx, Ly, Nx, Ny)
# set boundary conditions here
T_h = heat.solve()
```

In [9]:

```
class SteadyHeat2D:
    def __init__(self, Lx, Ly, dimX, dimY, if_point=False, point_i=0, poi
        self.l = Lx
        self.h = Ly
        self.dimX = dimX
        self.dimY = dimY
        self.if_point = if_point
        self.point_i = point_i
        self.point_j = point_j
        self.point_q = point_q

        self.dx = Lx / (dimX - 1)
        self.dy = Ly / (dimY - 1)

        self.A = np.zeros((self.dimX * self.dimY, self.dimX * self.dimY))
        self.b = np.zeros(self.dimX * self.dimY)

    # build the linear system
    def set_inner(self):
        for i in range(1,self.dimY-1):
```

```
            for j in range (1,self.dimX-1):
                k = i * self.dimX + j
                self.A[k, k] = -2 * (1 / self.dx**2 + 1 / self.dy**2)
                self.A[k, k - 1] = 1 / self.dx**2
                self.A[k, k + 1] = 1 / self.dx**2
                self.A[k, k - self.dimX] = 1 / self.dy**2
                self.A[k, k + self.dimX] = 1 / self.dy**2
                self.b[k] = 0

    # set the boundary conditions
    # south
    def set_south(self, bc_type, T_d=0.0, q=0.0, alpha = 0.0, T_inf=0.0):

        for j in range(self.dimX):
            k = j
            self.A[k, :] = 0
            self.b[k] = 0

            if bc_type == 'D':
                self.A[k, k] = 1
                self.b[k] = T_d

            elif bc_type == 'N':
                self.A[k, k] = -3 / 2 / self.dy
                self.A[k, k + self.dimX] = 2 / self.dy
                self.A[k, k + 2 * self.dimX] = - 1 / 2 / self.dy
                self.b[k] = q

            elif bc_type == 'R':
                self.A[k, k] = -3 / 2 / self.dy + alpha
                self.A[k, k + self.dimX] = 2 / self.dy
                self.A[k, k + 2 * self.dimX] = - 1 / 2 / self.dy
                self.b[k] = alpha * T_inf
            else:
                raise ValueError('Unknown boundary condition type')

    # north
    def set_north(self, bc_type, T_d=0.0, q=0.0, alpha = 0.0, T_inf=0.0):

        for j in range(self.dimX):
            k = (self.dimY - 1) * self.dimX + j
            self.A[k, :] = 0
            self.b[k] = 0

            if bc_type == 'D':
                self.A[k, k] = 1
                self.b[k] = T_d

            elif bc_type == 'N':
                self.A[k, k] = 3 / 2 / self.dy
                self.A[k, k - self.dimX] = - 2 / self.dy
                self.A[k, k - 2 * self.dimX] = 1 / 2 / self.dy
                self.b[k] = q

            elif bc_type == 'R':
                self.A[k, k] = 3 / 2 / self.dy + alpha
                self.A[k, k - self.dimX] = - 2 / self.dy
                self.A[k, k - 2 * self.dimX] = 1 / 2 / self.dy
                self.b[k] = alpha * T_inf
            else:
```

```
            raise ValueError('Unknown boundary condition type')

    # west
    def set_west(self, bc_type, T_d=0.0, q=0.0, alpha = 0.0, T_inf=0.0):

        for i in range(self.dimY):
            k = i * self.dimX
            self.A[k, :] = 0
            self.b[k] = 0

            if bc_type == 'D':
                self.A[k, k] = 1
                self.b[k] = T_d

            elif bc_type == 'N':
                self.A[k, k] = -3 / 2 / self.dx
                self.A[k, k + 1] = 2 / self.dx
                self.A[k, k + 2] = - 1 / 2 / self.dx
                self.b[k] = q

            elif bc_type == 'R':
                self.A[k, k] = -3 / 2 / self.dx + alpha
                self.A[k, k + 1] = 2 / self.dx
                self.A[k, k + 2] = - 1 / 2 / self.dx
                self.b[k] = alpha * T_inf
            else:
                raise ValueError('Unknown boundary condition type')

    # east
    def set_east(self, bc_type, T_d=0.0, q=0.0, alpha = 0.0, T_inf=0.0):

        for i in range(self.dimY):
            k = i * self.dimX + self.dimX - 1
            self.A[k, :] = 0
            self.b[k] = 0

            if bc_type == 'D':
                self.A[k, k] = 1
                self.b[k] = T_d

            elif bc_type == 'N':
                self.A[k, k] = 3 / 2 / self.dx
                self.A[k, k - 1] = - 2 / self.dx
                self.A[k, k - 2] = 1 / 2 / self.dx
                self.b[k] = q

            elif bc_type == 'R':
                self.A[k, k] = 3 / 2 / self.dx + alpha
                self.A[k, k - 1] = - 2 / self.dx
                self.A[k, k - 2] = 1 / 2 / self.dx
                self.b[k] = alpha * T_inf
            else:
                raise ValueError('Unknown boundary condition type')


    # solve the linear system
    def solve(self):
        self.set_inner()
        if self.if_point:
            k = self.point_i * self.dimX + self.point_j
```

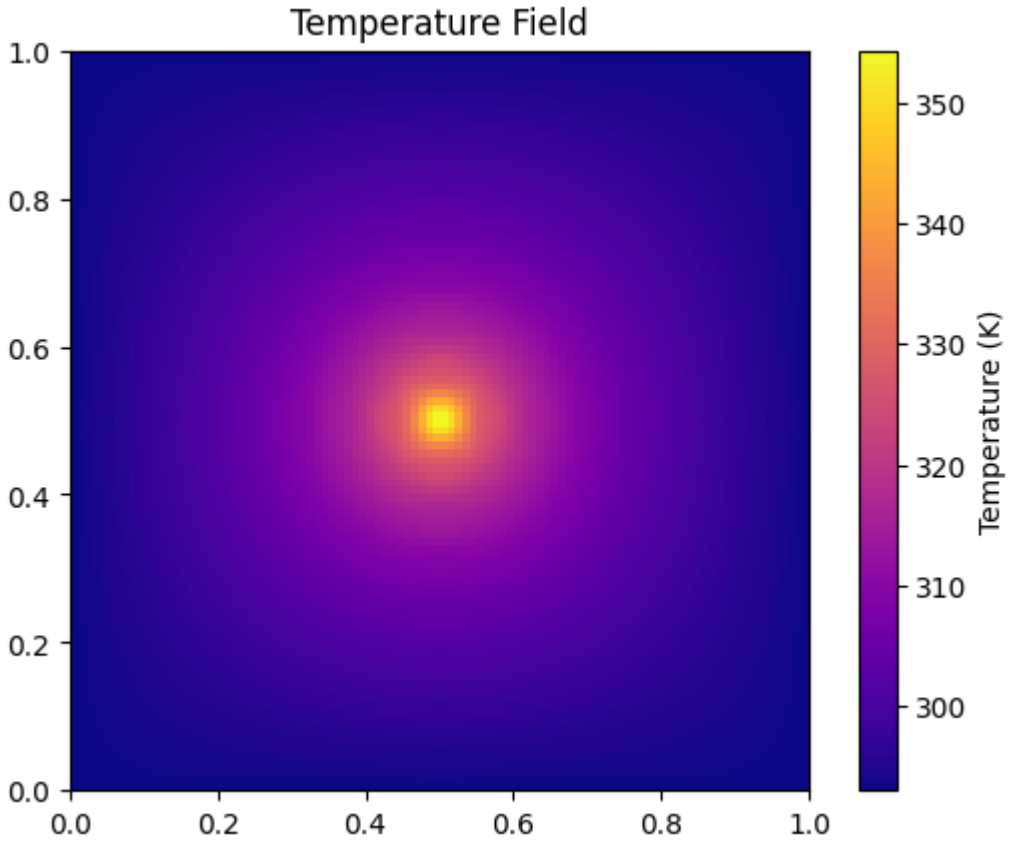
```
            self.b[k] = - self.point_q
        self.T = np.linalg.solve(self.A, self.b)
        self.T = self.T.reshape(self.dimY, self.dimX)
        return self.T
```

In [10]:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import cm

# ------------------------------------------------------
# Domain & mesh
# ------------------------------------------------------

# Your code

# Source location and strength
xs, ys = 0.5, 0.5
q = 100.0
T_d = 293.0

x = np.linspace(0, Lx, Nx)
y = np.linspace(0, Ly, Ny)
X, Y = np.meshgrid(x, y)

# ------------------------------------------------------
# Stage 3: 2D Green functions (X11 and X12), compute them using stage 3
# ------------------------------------------------------
G11 = G_2D_X11(X, Y, xs, ys, Lx, Ly) * q
G11 += T_d

G12 = G_2D_X12(X, Y, xs, ys, Lx, Ly) * q
G12 += T_d

# # ------------------------------------------------------
# # Finite difference solver (homogeneous part)
# # ------------------------------------------------------

# # Case 1: X11 = Dirichlet–Dirichlet–Dirichlet–Dirichlet
# heat1 = SteadyHeat2D(Lx, Ly, Nx, Ny)
# heat1.set_south('D', T_d=T_d)
# heat1.set_north('D', T_d=T_d)
# heat1.set_east('D',  T_d=T_d)
# heat1.set_west('D',  T_d=T_d)

# # Case 2: X12 = Dirichlet–Neumann BC
# # Example:
# # South  = Dirichlet
# # West   = Dirichlet
# # North  = Neumann (zero flux)
# # East   = Neumann (zero flux)

# heat2 = SteadyHeat2D(Lx, Ly, Nx, Ny)
# heat2.set_south('D', T_d=T_d)
# heat2.set_west('D',  T_d=T_d)
# heat2.set_north('N', q=0.0)
# heat2.set_east('N',  q=0.0)

# ------------------------------------------------------
# Plots
# ------------------------------------------------------
```

```
print("Dirichlet–Dirichlet Green function:")
plot(G11)
print("Dirichlet–Neumann Green function:")
plot(G12)
```

```
Dirichlet–Dirichlet Green function:
```




```
Dirichlet–Neumann Green function:
```